InfluenceIQ: implementation and operating guide

Status date: 19 October 2026. Canonical project: `/home/amitkumar/Music/MAI Project`. This is a research prototype, not a validated measure of a person's influence. The website runs inference on the local laptop. The Android application has a separate native processing implementation; website benchmark scores do not describe Android accuracy.

1. What is implemented

Area	Implementation and evidence	Important boundary
Accounts	Local registration/login, password hashing, authenticated ownership checks, account-scoped report transfer	Same account requires the same configured backend; offline devices do not synchronise automatically
Capture	Browser microphone or shared-tab audio, pause/resume, IndexedDB recording recovery	HTTPS or localhost required; keep the browser open
Live text	AudioWorklet captures 15-second PCM windows; authenticated local preview endpoint, one inference worker, bounded backpressure	Draft only; no live speaker tracking; incomplete final window is covered by saved-recording analysis
Upload	WAV, MP3, M4A, MP4, FLAC, OGG, WebM, AAC, OPUS, WMA, AIFF/AIF, AMR, 3GP through FFmpeg	Codec support depends on installed FFmpeg; multipart order is explicit
Transcription	Local faster-whisper, CPU int8, automatic/English/Hindi settings, vocabulary hints, timestamped word cache	Recognition errors remain; language tags are not token-level code-switch annotations
Speakers	Local Sherpa-ONNX segmentation/embedding/clustering; exact count optional; automatic rerun when count unknown	Cluster IDs are not personal identities; arbitrary voices are not automatically named
People	Editable names/roles; explicit self-introduction suggestions; shared transcript/evidence labels	Name suggestions and roles require human review

Area	Implementation and evidence	Important boundary
Presentations	PPTX text/tables/notes/metadata; optional local image OCR; temporal/semantic matching, TF-IDF option	Binary PPT must be converted; OCR is disabled by default; no observed slide timings means temporal evidence is limited
NLP	English/Hindi/romanised-Hindi rules, event candidates, evidence links, decision chains, action review	No generative summaries/chat/Q&A; candidate extraction is heuristic
Influence	Seven weighted, normalised contribution features with source evidence	Not causal influence, personality, employment suitability, or an independently validated ranking
Progress	Upload percentage, stage progress, elapsed time, comparable-run ETA, automatic result refresh	Overall percentage and ETA are estimates; unavailable ETA remains unavailable
Exports	Structured and human-readable reports, transcript export, local account-scoped sync	Review exported personal information before sharing
Android	Native Java/C++ app, local speech/speaker models, storage and analysis; Java tests and APK build	Physical Samsung testing, battery and thermal evaluation remain required

2. Main code map

- `backend/api.py`: authentication, ownership, upload validation, durable scheduling, preview API and report edits.
- `frontend/src/App.jsx`: capture, upload, meeting history, progress and review interface.
- `frontend/src/livePreview.js`, `frontend/public/recording-preview.js`: bounded live recording previews.
- `src/audio/`: decoding, VAD, ASR, word observations, diarization and speaker attribution.
- `src/pipeline/orchestrator.py`: complete offline pipeline and progress stages.
- `src/ppt/`, `src/fusion/`: presentation extraction and alignment.
- `src/meeting/`: event, evidence, interaction, decision and influence logic.
- `src/evaluation/`: metrics, baselines and ablations; missing references explicitly yield unavailable metrics.
- `tests/`, `scripts/test_browser_offline.cjs`, `android/test.sh`: functional regression coverage.
- `config/config.yaml`: model, thresholds, language, scoring and alignment settings.

3. Run commands

These are individual terminal commands, not a script to install. Existing model files and the `.venv` are already provisioned. Do not start a second backend against the same database/runtime. Ports 8000 and 5173 can belong to unrelated services.

Terminal 1, if the backend is not already running:

```
cd "/home/amitkumar/Music/MAI Project"
export HF_HUB_OFFLINE=1
export TRANSFORMERS_OFFLINE=1
export HF_HUB_DISABLE_TELEMETRY=1
export OMP_NUM_THREADS=4
export MAI_HOST=127.0.0.1
export MAI_PORT=8001
.venv/bin/python -m backend.api
```

Terminal 2, development website:

```
cd "/home/amitkumar/Music/MAI Project/frontend"
export MAI_BACKEND_URL=http://127.0.0.1:8001
npm run dev -- --host 127.0.0.1 --port 5174 --strictPort
```

Open `http://127.0.0.1:5174`. A production build is also served by the backend after `npm run build`. Keep all inference local; Internet is needed only for initial installation/model or public evaluation-data provisioning. Optional account/report transfer needs connectivity to the same trusted server, not cloud speech processing.

4. Test the website manually

1. Register/login and confirm another account cannot view the first account's meetings.
2. Record at least 35 seconds. Check live drafts, pause for five seconds, resume, and stop. Recover the locally saved recording. Drafts have no speaker labels; full analysis must still cover every recorded section.
3. Upload an audio file and PPTX. Replace/remove the PPTX; filename should appear once. Start analysis and check elapsed time, progress, draft updates and automatic completed tabs without refreshing.
4. Search a known transcript word. Check highlight counts, speaker filter and seek-to-audio evidence.
5. Edit a participant name and role. Verify transcript, evidence, participants and export use those labels.
6. If speakers are fragmented, open Overview → Review speaker separation. Choose automatic rerun when count is unknown; use an exact count only when verified. Review names again after any rerun.
7. Review action and decision candidates against audio, including negations and ambiguous assignments. A zero influence score means no scored contribution was detected, not that the person had no influence.
8. Test with Wi-Fi disabled after loading the local website. Confirm recording/analysis still works; internet account services are not required. Local loopback is still used by the web interface.

5. Test and evaluation commands

Run from the project root, preferably with no other inference jobs for reliable performance timing.

```
.venv/bin/python -m pytest -c config/pytest.ini -q
```

Frontend directory:

```
npm run lint
npm run build
```

Project root:

```
node scripts/test_browser_offline.cjs
./android/test.sh
./android/build.sh
.venv/bin/python scripts/evaluate_research_audio.py
.venv/bin/python scripts/evaluate_indic_pilot.py
.venv/bin/python scripts/evaluate_indic_pilot.py --model-size medium
```

The research audio script reuses saved diarization observations. ASR caches also avoid repeat inference. Cached reruns are functional checks, not fresh latency measurements. Preserve the original JSON evidence; use a separate evaluation directory and remove only that experiment's caches to measure a new cold run. Do not delete user recordings or the main database.

For a meeting with independently prepared annotation JSON:

```
.venv/bin/python scripts/run_evaluation.py --meeting-id YOUR_MEETING_ID --annotations
YOUR_ANNOTATIONS.json --ablation --output-dir outputs/evaluation/YOUR_RUN
```

The public pilot manifests contain local file paths, dataset revisions, selection rules and hashes. Audio stays under `.runtime/research-evaluation/`. Read `ANNOTATION_AND_SUBMISSION.md` before turning these pilot results into paper claims.

6. Performance and training decisions

The ASR word cache allows speaker corrections without decoding the same speech again. A single worker prevents competing neural jobs inside the API. The live preview queue is bounded and yields to full analysis. The offline diarization engine still loads the complete waveform to keep global speaker identities; it is not a constant-memory streaming diarizer. Normalised WAV files remain necessary for parts of the current pipeline. Do not claim that all intermediate audio or heavy dependencies have been eliminated.

Use the small model for the existing latency trade-off; medium is already installed and can be selected in Settings. Compare measured accuracy and latency on your own labelled recordings before changing defaults. No fine-tuning is required to run. Domain fine-tuning would require consented, transcribed training recordings and separate development/test speakers. Increasing model size or fine-tuning cannot guarantee perfect Hindi/Hinglish

recognition.

7. Remaining release gates

- Independent multi-speaker Hindi/Hinglish meeting annotations and a substantially larger held-out evaluation.
- Human ratings for influence, decisions, action items and slide alignment; inter-rater agreement and adjudication.
- Long-duration physical Samsung tests, background/lock-screen behavior, battery, heat and interruption recovery.
- Human author review, affiliations, funding/conflict/ethics declarations and final manuscript submission approval.
- Larger calibration study before treating the chosen speaker threshold as general-purpose. Existing 139-group reports require rerun and review; they are not silently rewritten.

No claimed feature parity with read.ai, patent novelty, publication acceptance, or perfect accuracy follows from passing functional tests.